IMPLEMENTATION ROADMAP · MVP BACKEND

RaithaMarga

Java Spring Boot + Firebase Backend Guide

A plain-language, step-by-step guide to build the REST API that powers the live RaithaMarga frontend.

Stack: React → Spring Boot → Firebase Auth + Firestore
Host: Vercel (frontend) · Render (backend)

MVP SCOPE IN THIS GUIDE

Login · Farmer & Buyer profiles · Listings · Requirements

Matching · Deals (status flow) · Render deployment

Removed from MVP: Weigh-proof photos / proofs API

Live frontend · https://raitha-marga.vercel.app/

1. What this guide is

RaithaMarga is a farmer-to-buyer marketplace. The live website is already on Vercel: **https://raitha-marga.vercel.app/**. This document tells you how to build the **Java Spring Boot backend** that the React frontend will call.

Read it like a recipe. Do not try to cook the whole meal on day one. Finish one milestone, test it with the frontend, then move to the next.

Live product promise (from the website)

Sell Direct. Sell Together. Sell with Trust. Small and marginal farmers list produce, nearby lots can be combined, and verified buyers confirm deals at a transparent price. Kannada is the default language; English is available from the header.

Important change from the older plan

The website currently mentions **weigh-proof photos** (timestamped scale photos at pickup/delivery). You asked to **remove weigh-proof photos from the frontend**. This guide therefore **does not include** ProofController, ProofService, the `proofs` collection, Firebase Storage for evidence photos, or `POST /api/proofs`. Deals complete with status changes only.

Who should use this

Anyone on the team who will write Java, wire Firebase, or deploy the API. You do not need to memorise every class on day one. Follow the order in section 6.

What you will have when you finish

- A Spring Boot 21 REST API running locally on port 8080.
- Firebase Authentication for login (no homemade passwords).
- Cloud Firestore holding users, farmer/buyer profiles, listings, requirements, matches and deals.
- The React frontend sending a Firebase ID token on every protected call.
- The same API deployed on Render (free plan is fine for MVP testing).

What you will not build in this MVP

- Weigh-proof / scale photos and a proofs API.
- Payments / UPI settlement inside the backend.
- A custom password database.
- SQLite or Express — those were earlier ideas and are not part of this stack.
- Advanced analytics dashboards.

2. Big picture — how the pieces talk

Think of three layers. The browser never talks to Firestore with admin power. The browser talks to Spring Boot. Spring Boot checks the login token, applies business rules, then reads or writes Firestore.

Layer	What it is	What it does
Frontend	React app on Vercel raitha-marga.vercel.app	Login, farmer/buyer screens, Add Produce, browse, deal status. Sends JSON + Bearer token.
API	Java 21 + Spring Boot localhost:8080 or Render	Verifies Firebase ID token. Owns roles, matching rules and deal state changes.
Auth	Firebase Authentication	Email/password sign-up and sign-in. Issues the ID token. Stores no RaithaMarga business data.
Database	Cloud Firestore	users, farmers, buyers, listings, requirements, matches, deals.

Figure — request path from a farmer tapping “Add Produce” to a row in Firestore.

Request path in one sentence

React signs the user in with Firebase → React attaches `Authorization: Bearer <idToken>` → Spring filter verifies the token with the Admin SDK → UID + role are loaded from `users/{uid}` → the controller runs only if that role is allowed → Firestore is updated → JSON is returned to the UI.

> **Do not invent a second login system**
>
> If Firebase Authentication is on, Firestore must never store plaintext passwords. The backend only stores profile and role data. The browser must never be trusted for farmerId, buyerId or role — those come from the verified token + Firestore.

How the live website maps to APIs

What the farmer / buyer sees	Backend endpoint to build
Login / Get Started (Firebase email-password)	No custom /login API. Frontend uses Firebase SDK. Backend exposes GET /api/me.
I’m a Farmer — profile + land / village details	POST /api/farmers/profile
I’m a Buyer — business + district	POST /api/buyers/profile
List in seconds — crop, quantity, expected price	POST /api/listings and GET /api/listings
Get combined & matched — same crop, nearby lots	GET /api/matches (plus requirements)
Confirm with a verified buyer — fair price, trust	POST /api/deals
Deliver and complete the sale (no photo proof)	PATCH /api/deals/{id}/status

3. Project folder structure

Create one Maven project named `raithamarga-backend`. Keep names boring and predictable so four developers can work without colliding.

```
raithamarga-backend/
  pom.xml
  Dockerfile
  .gitignore
  .env.example
  README.md
  src/main/java/com/raithamarga/backend/
    RaithaMargaApplication.java
    config/     FirebaseConfig.java  SecurityConfig.java  CorsConfig.java
    controller/ UserController  FarmerController  BuyerController
                ListingController  RequirementController
                MatchController  DealController
    service/    UserService  ListingService
                MatchingService  DealService
    repository/ FirestoreRepository.java
    model/      User  FarmerProfile  BuyerProfile
                ProduceListing  BuyerRequirement  Match  Deal
    security/   FirebaseAuthenticationFilter.java
    dto/        (request/response objects)
    exception/  (API error handlers)
  src/main/resources/application.yml
```

Why Proof* classes are missing

ProofController, ProofService, Proof.java and the proofs collection were in the older roadmap because of weigh-proof photos. They are intentionally left out. Do not add empty stubs “just in case” — they will confuse the team.

4. Firebase setup (do this before writing Java)

The frontend already expects Firebase. The backend uses a **service account** so it can verify tokens and read/write any collection. Never put that JSON file in Git.

1 Create or select the Firebase project

Use one project named RaithaMarga for both the Vercel frontend and the Spring Boot API.

2 Turn on Authentication

Enable Email/Password only for MVP. Firebase keeps the passwords. You do not.

3 Create Cloud Firestore

Start in test mode only on a throwaway project. For the real project, lock rules so only the Admin SDK (the server) writes business data.

4 Create a service account

Project settings → Service accounts → Generate new private key. Save the JSON on your laptop outside the repo.

5 Keep secrets out of Git

Locally use Application Default Credentials or a file that is gitignored. On Render, paste values into environment secrets.

5. Firestore collections (MVP data model)

Each collection is a folder of documents. The document id is shown after the slash. Use the Firebase UID as the document id for users, farmers and buyers so one login maps to one profile.

Collection	Document fields	Who writes it
users/{uid}	name, email, role (FARMER or BUYER), status, createdAt	Created on first successful /api/me or profile save.
farmers/{uid}	village, taluk, district, state, landAcres, verificationStatus	Farmer, via POST /api/farmers/profile.
buyers/{uid}	businessName, district, verificationStatus	Buyer, via POST /api/buyers/profile.
listings/{listingId}	farmerId, crop, quantity, unit, grade, price, location, availabilityDate, status	Farmer. farmerId is forced to the logged-in UID.
requirements/{id}	buyerId, crop, quantity, quality, location, targetPrice	Buyer. buyerId is forced to the logged-in UID.
matches/{matchId}	listingId, requirementId, score, reasons, status	Server only. MatchingService writes these.
deals/{dealId}	farmerId, buyerId, listingId, quantity, price, status, timestamps	Created when a buyer shows interest. Status changes via PATCH.

There is **no proofs collection** in this MVP. Deal completion is a status change, not a photo upload.

Deal statuses you must support

INTERESTED → **CONFIRMED** → **IN_PROGRESS** → **COMPLETED**. Also allow **CANCELLED** and **FAILED** when a party walks away or a lot is no longer available. Validate every jump on the server. A client must not be allowed to jump from INTERESTED straight to COMPLETED.

6. Authentication — the only flow you need

If this flow works, every later API becomes easy. If this flow is sloppy, every later API is unsafe. Build it first.

#	Where	What happens
1	**React**	User taps Login / Get Started. Firebase signIn or signUp with email and password.
2	**Firebase Auth**	Returns an ID token (JWT) to the browser.
3	**React**	Calls GET /api/me with header Authorization: Bearer <token>.
4	**Spring filter**	FirebaseAuthenticationFilter reads the header.
5	**Admin SDK**	Verifies the token. Extracts the Firebase UID.
6	**Firestore**	Loads users/{uid}. Reads role = FARMER or BUYER.
7	**Controller**	Allows the operation only if the role matches the API.

Frontend call pattern (use this everywhere)

```
const token = await user.getIdToken();

await fetch(`${import.meta.env.VITE_API_BASE_URL}/api/me`, {
  headers: { Authorization: `Bearer ${token}` }
});
```

After Render gives you a URL, put it in the frontend as `VITE_API_BASE_URL` and redeploy Vercel. Until then use `http://localhost:8080`.

7. Build order — do not skip steps

This is the same order as the original roadmap, with proofs removed and the live frontend used as the acceptance test.

1 Create the Maven project in VS Code

Use Spring Initializr. Java 21. Packaging JAR. Dependencies: Spring Web, Validation, Spring Boot Test.

2 Add Firebase Admin SDK

In pom.xml add com.google.firebase:firebase-admin. Do not commit the service-account JSON.

3 GET /api/health

Return {"status":"ok"}. Prove the process starts on port 8080.

4 FirebaseConfig + Firestore

One @Bean that initialises FirebaseApp once and returns FirestoreClient.getFirestore().

5 Token verification filter

Reject missing/invalid Bearer tokens on every route except /api/health.

6 GET /api/me

Return the Firestore user + role. This is Milestone 1. Test it from the Vercel login.

7 Farmer and buyer profiles

POST /api/farmers/profile and POST /api/buyers/profile. UID comes from the token.

8 Listing APIs

POST /api/listings, GET /api/listings, GET /api/listings/{id}. Connect the Add Produce form.

9 Requirements + browse

Buyers post demand. Farmers/buyers can filter by crop and location.

10 Deterministic matching

Score on crop + quantity + location + price/quality. Write matches/{id}.

11 Deals

POST /api/deals and PATCH /api/deals/{id}/status with the four-step lifecycle.

12 Hardening

Validation, role checks, CORS for the Vercel origin, error JSON, a few tests.

13 Deploy

Docker + Render. Point VITE_API_BASE_URL at the onrender.com URL.

8. Core REST APIs

Keep paths short and predictable. JSON in, JSON out. Every path below `/api/health` requires a valid Firebase token.

Method	Path	Who	Purpose
GET	**/api/health**	Anyone	Liveness for local run and Render health checks.
GET	**/api/me**	Logged-in user	Return profile + role for the dashboard switch.
POST	**/api/farmers/profile**	FARMER	Create or update village, taluk, acres.
POST	**/api/buyers/profile**	BUYER	Create or update business name and district.
POST	**/api/listings**	FARMER	Add produce: crop, qty, unit, grade, price, date.
GET	**/api/listings**	Logged-in	Browse / filter listings.
GET	**/api/listings/{id}**	Logged-in	One listing detail.
POST	**/api/requirements**	BUYER	Buyer demand for a crop.
GET	**/api/matches**	Logged-in	Matches for the caller’s listings or requirements.
POST	**/api/deals**	BUYER	Open a deal from a listing / match.
PATCH	**/api/deals/{id}/status**	Both parties	Move INTERESTED → … → COMPLETED.

Not in this MVP

POST /api/proofs is removed. Do not accept evidenceUrl uploads. If the old frontend still posts a photo field, ignore it or return 410 Gone so the UI team knows to drop the control.

Matching rule (keep it boring and testable)

Do not start with machine learning. A match is good when all of these are true:

- **Crop** is the same (normalise spelling: “ragi” and “Ragi” are one crop).
- **Quantity** on the listing is at least the requirement, or can be combined later as a follow-up.
- **Location** is the same district for MVP (taluk is a bonus filter).
- **Price** on the listing is less than or equal to the buyer’s targetPrice, within a small tolerance you pick (for example 10%).
- **Quality / grade** meets or beats what the buyer asked.

Store a short `reasons` array on the match document ("same crop", "same district", "price within 8%") so the UI can show why it matched.

9. Local development in VS Code

Tools to install once

- JDK 21 (Temurin is fine).
- Maven 3.9+.
- VS Code with Extension Pack for Java and Spring Boot Extension Pack.
- A Firebase project and the service-account JSON stored **outside** the repo.

application.yml

```
server:
  port: ${PORT:8080}

raithamarga:
  cors:
    allowed-origins: https://raitha-marga.vercel.app, http://localhost:5173
```

Commands

```
cd raithamarga-backend
mvn clean package
mvn spring-boot:run

# then in a browser or curl
curl http://localhost:8080/api/health
```

You should see a tiny JSON health payload. If the process dies, it is almost always missing Firebase credentials — fix that before writing controllers.

FirebaseConfig idea (not copy-paste production code)

```
@Configuration
public class FirebaseConfig {

    @Bean
    public Firestore firestore() throws Exception {
        GoogleCredentials credentials =
            GoogleCredentials.getApplicationDefault();

        FirebaseOptions options = FirebaseOptions.builder()
            .setCredentials(credentials)
            .build();

        if (FirebaseApp.getApps().isEmpty()) {
            FirebaseApp.initializeApp(options);
        }
        return FirestoreClient.getFirestore();
    }
}
```

Locally, point Google Application Default Credentials at your service account (`export GOOGLE_APPLICATION_CREDENTIALS=/absolute/path/key.json`). On Render, prefer environment variables (project id, client email, private key) and build credentials in code from those values so you never upload the JSON file.

CORS — otherwise the Vercel site cannot call you

Browsers block a Vercel page from calling localhost or onrender.com unless the API sends the right CORS headers. Allow the live origin and your local Vite origin. Allow the Authorization header. Allow GET, POST, PATCH, OPTIONS.

10. Free deployment on Render

For an MVP, Render's free Web Service is enough. Know the limit: a free instance **sleeps after about 15 minutes of no traffic** and can take around a minute to wake. That is annoying in a demo — hit /api/health once before you show the app. Render itself says free instances are not for real production.

1. **Push to GitHub**

   Commit the backend (without secrets). Create a public or private repo.

2. **New Web Service**

   Render dashboard → New → Web Service → pick the repo.

3. **Use Docker**

   Docker runtime gives a predictable Java 21 JRE. Do not fight native Maven buildpacks unless you must.

4. **Free plan**

   Choose Free for testing.

5. **Environment / secrets**

   Set PORT, FIREBASE_PROJECT_ID, FIREBASE_CLIENT_EMAIL, FIREBASE_PRIVATE_KEY.

6. **Health check**

   Path = /api/health so Render knows the JVM is up.

7. **Deploy and copy the URL**

   You get https://something.onrender.com.

8. **Point the frontend**

   Set VITE_API_BASE_URL to that URL on Vercel and redeploy https://raitha-marga.vercel.app/.

Dockerfile

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/*.jar app.jar
ENV PORT=8080
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

Build the JAR first with `mvn clean package`. If Render builds from source, add a Docker build that runs Maven inside the image, or use a two-stage Dockerfile. The important rule: the process must listen on `$PORT`.

Render environment variables

Name	Example / note
PORT	8080 — Render also injects this. Spring must read ${PORT:8080}.
FIREBASE_PROJECT_ID	Your Firebase project id.
FIREBASE_CLIENT_EMAIL	Service account email.
FIREBASE_PRIVATE_KEY	The private key. Keep the \n newlines. Mark as secret.

Never commit these values. If a key leaks in a screenshot or chat, rotate the service account.

11. Security checklist

- Never store plaintext passwords. Firebase Authentication owns that.
- Never commit service-account JSON, .env, or private keys.
- Verify the Firebase ID token on every protected route.
- Do not trust farmerId, buyerId or role sent by the browser. Take UID from the token.
- A farmer may write only their own listings. A buyer may write only their own requirements.
- Validate crop, quantity > 0, price > 0, and legal status transitions.
- Use HTTPS only in production (Vercel and Render already do).
- Do not log access tokens, private keys, or raw Authorization headers.
- Lock Firestore security rules so random browsers cannot write collections directly.
- Do not accept photo / evidence uploads for weigh-proof in this MVP.

12. Milestones — ship in four slices

Slice	User-visible result	Done when
M1 Identity	Login on the Vercel site opens a Farmer or Buyer home.	GET /api/me returns role from users/{uid}.
M2 List produce	Farmer submits crop, qty, price. It appears in My Listings.	POST /api/listings writes listings/{id} with farmerId = UID.
M3 Demand + match	Buyer posts a requirement. Matches appear for the same crop and district.	GET /api/matches returns scored rows with reasons.
M4 Deal lifecycle	Interest → confirm → in progress → completed (or cancelled).	PATCH /api/deals/{id}/status rejects illegal jumps.

There is no Milestone 5 for photos. Payment tracking on the marketing site can stay as a later P2 — do not block M4 on UPI.

13. Team split

Person	Owns
Developer 1	Spring Boot skeleton, FirebaseConfig, authentication filter, /api/health, /api/me, CORS.
Developer 2	Farmer profile + listing APIs + wiring the Add Produce form.
Developer 3	Buyer profile + requirements + MatchingService.
Developer 4	Deals, status machine, Render deploy, end-to-end test against the Vercel site.

14. What to do first tomorrow morning

1. **Install JDK 21, Maven, and the VS Code Java / Spring extensions.**
   Tick this off before you start the next number.

2. **Create raithamarga-backend with Spring Initializr (Web + Validation + Test).**
   Tick this off before you start the next number.

3. **Add the Firebase Admin SDK dependency.**
   Tick this off before you start the next number.

4. **Ship GET /api/health and open it in the browser.**
   Tick this off before you start the next number.

5. **Wire FirebaseConfig and confirm Firestore lists collections.**
   Tick this off before you start the next number.

6. **Implement token verification.**
   Tick this off before you start the next number.

7. **Test GET /api/me using a token from the live frontend login.**
   Tick this off before you start the next number.

8. **Only after that works, implement listing APIs. Do not start matching or deals on day one.**
   Tick this off before you start the next number.

15. Final stack (write this on the whiteboard)

Part	Choice
Frontend	Existing React app — https://raitha-marga.vercel.app/
Backend	Java 21 + Spring Boot
Authentication	Firebase Authentication (email / password)
Database	Cloud Firestore
File storage	Not required in this MVP (weigh-proof photos removed)
API style	REST / JSON
Local IDE	VS Code
Version control	GitHub
Frontend host	Vercel
Backend host	Render Web Service (Docker)

16. Honest limits of this PDF

This is a roadmap, not a zip file of a finished production backend. Code samples are there so you can see the shape of a class, not so you can paste them untested. Build and test each milestone against the real frontend before you start the next one.

> **If someone still asks “where is weigh proof?”**
>
> Answer: it was a marketing feature on the landing page. Product decided to drop scale photos from the first release to keep farmers moving from listing to deal without a camera step. The backend must not re-introduce proofs unless product puts the UI back.

RaithaMarga · Sell Direct. Sell Together. Sell with Trust.
Frontend https://raitha-marga.vercel.app/ · Backend Java 21 / Spring Boot / Firestore / Render